OTEL-04: Trace Instrumentation

> **Series:** OTEL — OpenTelemetry Integration | **Notebook:** 4 of 8 | **Created:** January 2026 | **Last Updated:** 01/28/2026

Instrumenting Applications for Distributed Tracing

Traces provide visibility into request flows across services. This notebook covers automatic and manual instrumentation techniques for popular languages with OpenTelemetry.

Table of Contents

Prerequisites

Requirement	Details
Knowledge	OTEL-01 and OTEL-03
Environment	Collector deployed and accessible

Requirement	Details
Language	Examples in Python, Java, Go, Node.js

1. Instrumentation Types

Comparison

Type	Effort	Customization	Coverage
Zero-code (auto)	Minimal	Limited	Framework-level
Manual	More	Full	Custom business logic
Hybrid	Medium	Balanced	Best of both

When to Use Each

Scenario	Recommendation
Quick start, standard frameworks	Auto-instrumentation
Custom business spans	Manual
Production with specific needs	Hybrid
Serverless / Lambda	SDK with auto-instrumentation

2. Auto-Instrumentation

Python Auto-Instrumentation

```
# Install
pip install opentelemetry-distro opentelemetry-exporter-otlp
opentelemetry-bootstrap -a install

# Run with auto-instrumentation
opentelemetry-instrument \
  --service_name my-python-app \
  --exporter_otlp_endpoint http://collector:4317 \
  python app.py
```

Java Auto-Instrumentation

```
# Download agent
curl -L -o opentelemetry-javaagent.jar \
  https://github.com/open-telemetry/opentelemetry-java-
instrumentation/releases/latest/download/opentelemetry-javaagent.jar

# Run with agent
java -javaagent:opentelemetry-javaagent.jar \
  -Dotel.service.name=my-java-app \
  -Dotel.exporter.otlp.endpoint=http://collector:4317 \
  -jar app.jar
```

Node.js Auto-Instrumentation

```
# Install
npm install @opentelemetry/auto-instrumentations-node \
  @opentelemetry/sdk-node \
  @opentelemetry/exporter-trace-otlp-http

# Run with auto-instrumentation
node --require @opentelemetry/auto-instrumentations-node/register app.js
```

```
// Or programmatically (tracing.js)
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-
instrumentations-node');
const { OTLPTraceExporter } = require('@opentelemetry/exporter-trace-otlp-
http');

const sdk = new NodeSDK({
  serviceName: 'my-node-app',
  traceExporter: new OTLPTraceExporter({
    url: 'http://collector:4318/v1/traces',
  }),
  instrumentations: [getNodeAutoInstrumentations()],
});

sdk.start();
```

3. Manual Instrumentation

Python Manual Tracing

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
OTLPSpanExporter
from opentelemetry.sdk.resources import Resource

# Setup
resource = Resource.create({"service.name": "my-python-app"})
provider = TracerProvider(resource=resource)
processor =
BatchSpanProcessor(OTLPSpanExporter(endpoint="http://collector:4317"))
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)

# Get tracer
tracer = trace.get_tracer(__name__)

# Create spans
@tracer.start_as_current_span("process_order")
def process_order(order_id):
    span = trace.get_current_span()
    span.set_attribute("order.id", order_id)

    with tracer.start_as_current_span("validate_order"):
        # Validation logic
        pass

    with tracer.start_as_current_span("charge_payment"):
        # Payment logic
        pass
```

Java Manual Tracing

```
import io.opentelemetry.api.GlobalOpenTelemetry;
import io.opentelemetry.api.trace.Span;
import io.opentelemetry.api.trace.Tracer;
import io.opentelemetry.context.Scope;

public class OrderService {
    private static final Tracer tracer =
GlobalOpenTelemetry.getTracer("order-service");

    public void processOrder(String orderId) {
        Span span = tracer.spanBuilder("process_order").startSpan();
        try (Scope scope = span.makeCurrent()) {
            span.setAttribute("order.id", orderId);

            validateOrder(orderId);
            chargePayment(orderId);
        } finally {
            span.end();
        }
    }
}
```

Go Manual Tracing

```go
import (
    "context"
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
)

var tracer = otel.Tracer("order-service")

func ProcessOrder(ctx context.Context, orderID string) error {
    ctx, span := tracer.Start(ctx, "process_order")
    defer span.End()

    span.SetAttributes(attribute.String("order.id", orderID))

    if err := validateOrder(ctx, orderID); err != nil {
        span.RecordError(err)
        return err
    }

    return chargePayment(ctx, orderID)
}
```

4. Context Propagation

W3C Trace Context

The standard propagation format:

Header	Example	Description
`traceparent`	`00-abc123-def456-01`	Trace ID, span ID, flags
`tracestate`	`dt=s:1`	Vendor-specific state

HTTP Propagation

Python (requests):

```python
from opentelemetry.propagate import inject
import requests

def call_downstream():
    headers = {}
    inject(headers)  # Adds traceparent, tracestate
    response = requests.get("http://downstream/api", headers=headers)
```

Extracting Context:

```python
from opentelemetry.propagate import extract

@app.route('/api/endpoint')
def endpoint():
    context = extract(request.headers)
    with tracer.start_as_current_span("endpoint", context=context):
        # Request handling
        pass
```

Messaging Propagation

Kafka:

```python
# Producer
from opentelemetry.propagate import inject

headers = []
inject(headers, setter=kafka_header_setter)
producer.send('topic', value=message, headers=headers)

# Consumer
context = extract(record.headers, getter=kafka_header_getter)
with tracer.start_as_current_span("process_message", context=context):
    process(record)
```

5. Span Attributes and Events

Adding Attributes

```
with tracer.start_as_current_span("checkout") as span:
    # Business attributes
    span.set_attribute("user.id", user_id)
    span.set_attribute("order.total", order_total)
    span.set_attribute("order.items", len(items))

    # Semantic convention attributes
    span.set_attribute("http.method", "POST")
    span.set_attribute("http.status_code", 200)
```

Adding Events

```
with tracer.start_as_current_span("payment") as span:
    span.add_event("payment_initiated", {"provider": "stripe"})

    result = process_payment()

    if result.success:
        span.add_event("payment_completed", {
            "transaction_id": result.transaction_id
        })
    else:
        span.add_event("payment_failed", {
            "error": result.error_message
        })
```

Recording Errors

```
from opentelemetry.trace import StatusCode

with tracer.start_as_current_span("operation") as span:
    try:
        result = risky_operation()
    except Exception as e:
        span.record_exception(e)
        span.set_status(StatusCode.ERROR, str(e))
        raise
```

6. Language-Specific Examples

Complete Python Example

```
# app.py
from flask import Flask, request
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import
OTLPSpanExporter
from opentelemetry.instrumentation.flask import FlaskInstrumentor
from opentelemetry.instrumentation.requests import RequestsInstrumentor
from opentelemetry.sdk.resources import Resource

# Setup
resource = Resource.create({"service.name": "checkout-api"})
provider = TracerProvider(resource=resource)
processor = BatchSpanProcessor(
    OTLPSpanExporter(endpoint="http://collector:4318/v1/traces")
)
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)

# Auto-instrument
app = Flask(__name__)
FlaskInstrumentor().instrument_app(app)
RequestsInstrumentor().instrument()

tracer = trace.get_tracer(__name__)

@app.route('/checkout', methods=['POST'])
def checkout():
    with tracer.start_as_current_span("process_checkout") as span:
        order = request.json
        span.set_attribute("order.id", order['id'])

        # Custom business span
        with tracer.start_as_current_span("validate_inventory"):
            validate_inventory(order['items'])

        return {"status": "success"}
```

```
// View spans with custom attributes
fetch spans, from:-1h
| filter isNotNull(order.id)
| fields timestamp, trace.id, span.name, order.id, duration
| sort timestamp desc
| limit 20
```

```
// Find spans with errors
fetch spans, from:-1h
| filter otel.status_code == "ERROR"
| fields timestamp, trace.id, span.name, otel.status_message
| sort timestamp desc
| limit 20
```

7. Best Practices

Span Naming

Good	Bad	Why
`HTTP GET /users/{id}`	`HTTP GET /users/123`	Low cardinality
`process_order`	`order_abc123`	Descriptive, stable
`db.query`	`SELECT * FROM...`	Operation, not data

Attribute Guidelines

Practice	Reason
Use semantic conventions	Standardization
Keep cardinality low	Query performance
Don't include PII	Security
Add business context	Debugging value

Performance Tips

Tip	Impact
Use batch processor	Reduces network calls
Sample appropriately	Reduces volume
Limit attribute size	Reduces payload
Don't over-instrument	Reduces noise

Summary

In this notebook, you learned:

- Instrumentation types: auto, manual, hybrid
- Auto-instrumentation for Python, Java, Node.js
- Manual span creation and management
- Context propagation across services
- Adding attributes and events to spans
- Language-specific implementation patterns
- Best practices for effective tracing

References

- OTel Python
- OTel Java
- OTel Go
- OTel Node.js

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.